



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA  
VICERRECTORADO ACADÉMICO  
COORDINACIÓN GENERAL DE PREGRADO  
PROYECTO DE CARRERA INGENIERÍA EN INFORMÁTICA  
CATEDRA: ESTADÍSTICAS II

## **ANÁLISIS SINTÁCTICO**

**Profesor:**  
Félix Márquez

**Autores:**  
Keibel Guilarte  
**C.I 28.726.602**  
Oriana Márquez  
**C.I 31.354.299**  
Jeanny Monagas  
**C.I 30.857.471**  
Victor Vargas  
**C.I 30.697.219**

**Ciudad Guayana, junio del 2026.**

Un **Árbol de Sintaxis Abstracta** (AST, por sus siglas en inglés, *Abstract Syntax Tree*) es una estructura de datos fundamental en las ciencias de la computación, modelada típicamente como un grafo dirigido acíclico, que representa la estructura lógica y semántica del código fuente de un programa informático. Es el subproducto principal de la fase de análisis sintáctico (parsing) dentro del ciclo de vida de un compilador o intérprete.

Para entender su naturaleza, es vital ubicar el AST dentro del proceso de compilación. Cuando una computadora lee código fuente, primero pasa por un **analizador léxico**, que convierte el texto plano en una secuencia de palabras clave y símbolos elementales llamados *tokens*. Posteriormente, el **analizador sintáctico** toma esta secuencia de tokens y construye el AST, validando que el código cumple con las reglas gramaticales del lenguaje.

Se le otorga el adjetivo "abstracto" porque destila el código hasta su esencia operativa, omitiendo deliberadamente todos los detalles sintácticos superficiales o de conveniencia visual que el lenguaje requiere para que el código sea legible por humanos. A diferencia de un *Árbol de Sintaxis Concreta* (o Parse Tree), que registra exhaustivamente cada coma, punto y coma, paréntesis de agrupación o espacio en blanco, el AST descarta esta "basura sintáctica" (syntax sugar). En el AST, la agrupación y la prioridad matemática no se representan con paréntesis, sino mediante la misma arquitectura del árbol (qué nodo es padre de cuál nodo).

En resumen, el AST es el puente crítico que transforma una cadena de texto bidimensional (el código fuente) en un modelo de datos estructurado y navegable que la máquina puede validar, optimizar y traducir a código ejecutable.

## 2. Características Principales y Propiedades

El diseño y la utilidad de un Árbol de Sintaxis Abstracta se fundamentan en una serie de características técnicas clave:

- **Estructura Jerárquica y Recursividad:** El árbol está compuesto por nodos interconectados con relaciones estrictas de padre e hijo. La raíz del árbol suele representar el programa completo o un bloque principal, mientras que las ramas desglosan funciones, declaraciones y expresiones. Esta estructura es inherentemente recursiva; un nodo de tipo "operación matemática" puede contener en su interior otros nodos que también sean "operaciones matemáticas", permitiendo modelar una complejidad infinita de sentencias anidadas.
- **Filtrado Estricto de "Ruido" Sintáctico:** El AST aísla la semántica (lo que el código hace) de la sintaxis (cómo se escribe). Caracteres delimitadores como llaves {}, corchetes [], o palabras clave redundantes de fin de bloque (como end o endif en ciertos lenguajes) desaparecen por completo. El simple hecho de que un conjunto de instrucciones sean nodos hijos de una declaración condicional if,

ya implica que pertenecen a ese bloque, haciendo innecesarios los delimitadores visuales.

- **Resolución Inherente de Ambigüedades:** En el texto plano, una expresión como  $A + B * C$  es ambigua a menos que el lenguaje defina reglas de precedencia. En el AST, esta ambigüedad ya ha sido resuelta matemáticamente por el analizador. La multiplicación  $B * C$  se agrupará obligatoriamente en un subárbol que actuará como uno de los operandos del nodo superior de suma. El AST garantiza una única interpretación posible del orden de ejecución.
- **Plataforma para Análisis Semántico:** El AST es la base sobre la cual los compiladores realizan el análisis semántico. Una vez construido, el compilador recorre el árbol para verificar cosas que la gramática básica no puede validar, como asegurarse de que no se intente sumar un texto con un número (comprobación de tipos) o que todas las variables utilizadas hayan sido declaradas previamente. A menudo, este proceso da lugar a un "AST Decorado", donde los nodos se enriquecen con información de tipos de datos.
- **Motor Ecosistémico de Herramientas de Desarrollo:** Más allá de los compiladores, el AST es la estructura que hace posible el desarrollo de software moderno. Es la estructura de datos que leen los "Linters" (como ESLint o SonarQube) para encontrar vulnerabilidades y malas prácticas; es lo que usan los formateadores automáticos (como Prettier) para reescribir el código con un estilo consistente; y es la base de los transpiladores (como Babel o TypeScript) para traducir código de un lenguaje a otro analizando su lógica profunda y no simplemente reemplazando palabras.

- **Ejemplo 1:  $3 + 4 * 2 - 1$**

BinOp('-')

BinOp('+')

Num(3.0)

BinOp('\*')

Num(4.0)

Num(2.0)

Num(1.0)

Nótese que el AST refleja la precedencia de operadores ( $*$  antes que  $+/-$ ) mediante la profundidad del árbol, sin necesidad de conservar paréntesis.

- **Ejemplo 2:**  $x = (1 + 2) * (3 - 4) / 5$

Assign('x')

BinOp('/')

BinOp('\*')

BinOp('+')

Num(1.0)

Num(2.0)

BinOp('-')

Num(3.0)

Num(4.0)

Num(5.0)

Aquí los paréntesis de la entrada no aparecen en el AST: su único efecto fue alterar el orden de evaluación, y ese orden ya quedó representado por la posición de los nodos en el árbol (evaluado:  $x = -0.6$ ).

## Comparación entre análisis LL y LR

| Criterio a evaluar             | Analizadores LL(k)                                                                                                                  | Analizadores LR(k)                                                                                                                             |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Enfoque de construcción</b> | <b>De arriba hacia abajo (Top-down):</b> Empiezan a armar el árbol desde la raíz principal hasta llegar a los detalles (las hojas). | <b>De abajo hacia arriba (Bottom-up):</b> Inician analizando las piezas más pequeñas (hojas) y las van uniendo hasta formar la raíz del árbol. |
| <b>Tipo de derivación</b>      | <b>Izquierda:</b> Siempre priorizan desglosar el elemento que se encuentra más hacia la izquierda.                                  | <b>Derecha inversa:</b> Desglosan el elemento situado más a la derecha, pero procesándolo de atrás hacia adelante.                             |
| <b>Mecanismo utilizado</b>     | Se basan en <b>llamadas recursivas</b> y tablas que predicen qué regla aplicar usando conjuntos lógicos (FIRST y FOLLOW).           | Utilizan una <b>pila de memoria junto con una máquina de estados</b> que ejecuta acciones de desplazamiento y reducción (shift-reduce).        |

|                                         |                                                                                                                               |                                                                                                                                            |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Manejo de recursividad izquierda</b> | <b>Prohibida:</b> Si la gramática la tiene, arrojará error. Es obligatorio modificar y reescribir las reglas para eliminarla. | <b>Totalmente admitida:</b> La manejan de forma natural y directa (ej. una expresión que se llama a sí misma a la izquierda).              |
| <b>Capacidad y alcance</b>              | <b>Más limitado:</b> Solo logran procesar un grupo más reducido y específico de gramáticas libres de contexto (GLC).          | <b>Mucho más potente:</b> Son capaces de entender e interpretar una variedad mucho más amplia de gramáticas sin ambigüedades.              |
| <b>Desarrollo a mano</b>                | <b>Muy sencillo:</b> Es bastante fácil de programar por un humano desde cero escribiendo el código directamente.              | <b>Muy complejo:</b> Hacer a mano las tablas de estados es casi imposible o muy tedioso, por lo que <b>exige usar software generador</b> . |
| <b>Ejemplos de herramientas</b>         | Implementaciones manuales recursivas, o analizadores basados en PEG (como el que usa Python en sus versiones recientes).      | Herramientas automáticas de generación como Bison, Yacc, CUP o creadores de parsers LALR/SLR.                                              |

Las estrategias de análisis sintáctico (LL y LR) no son solo conceptos teóricos; son la base tecnológica de los lenguajes y plataformas más utilizados en la actualidad. A continuación, se detalla cómo se aplican en entornos de producción masiva:

- **Python (Migración a PEG):** Un hito reciente muy relevante ocurrió a partir de la versión 3.9 de CPython. El equipo de desarrollo decidió reemplazar su histórico analizador LL(1) por un analizador PEG (*Parsing Expression Grammar*, un modelo recursivo descendente con capacidad de retroceso o *backtracking*). Esta decisión (documentada en el PEP 617) se tomó precisamente por las limitaciones técnicas del modelo LL(1) clásico para manejar la recursión izquierda y resolver reglas ambiguas con un solo token de anticipación (*lookahead*). Esto demuestra la evolución continua de los parsers descendentes en lenguajes de alto nivel.
- **Adopción de ANTLR (Estrategia LL adaptativa):** El generador de parsers ANTLR, que utiliza una estrategia avanzada LL(\*), es el motor detrás de sistemas de escala global. Es utilizado por Twitter para procesar más de 2 mil millones de consultas diarias, y forma parte de la infraestructura de Apache Hive,

Hadoop (Pig), Oracle SQL Developer, NetBeans (para analizar C++) e Hibernate (para el lenguaje HQL).

- **El Estándar C/C++ (Estrategia LR):** Por otro lado, herramientas como Bison y Yacc, que son generadores de parsers LALR(1) (un subconjunto altamente eficiente de la familia LR), continúan siendo el estándar indiscutible para la creación de compiladores de lenguajes de propósito general, estando sólidamente documentados como la referencia oficial en el ecosistema de software GNU.

### En síntesis (Tendencia de Uso):

Los lenguajes que poseen gramáticas más sencillas y deterministas, o aquellos donde se prioriza la facilidad de leer y mantener un parser programado a mano, tienden a decantarse por el **enfoque descendente (LL/PEG)**. Por el contrario, los lenguajes con gramáticas altamente complejas y que requieren el máximo rendimiento de un autómata basado en tablas de estados, optan por el **enfoque ascendente (LR/LALR)**.

### Implementación y Comparativa Práctica (Caso Lenguaje "L")

Para ilustrar de forma tangible las diferencias descritas en secciones anteriores, se desarrolló una prueba práctica implementando un mismo lenguaje "L" (enfocado en expresiones aritméticas básicas con +, -, \*, / y paréntesis) utilizando ambas arquitecturas de análisis.

El objetivo fue observar cómo operan ambas estrategias al procesar exactamente las mismas entradas de código:

#### Analizador Descendente (LL Parser)

Se implementó un parser LL(1) mediante la técnica **recursiva descendente**.

- **Adaptación de la Gramática:** Para que esto fuera posible, se tuvo que eliminar la recursión izquierda intrínseca de las matemáticas, transformando las reglas a un formato compatible (ej.  $E \rightarrow T E'$  ;  $E' \rightarrow + T E' \mid - T E' \mid \epsilon$ ).
- **Comportamiento:** El sistema imprime una traza de derivación que evidencia claramente cómo el árbol lógico se construye desde la raíz principal hacia los nodos inferiores (hojas)

#### Analizador Ascendente (LR Parser)

Se construyó un parser de tipo **shift-reduce** (desplazar y reducir) guiado por la precedencia natural de los operadores matemáticos.

- **Gramática:** A diferencia del modelo LL, este enfoque procesó la gramática en su forma natural, admitiendo sin problemas la recursión izquierda (ej.  $E \rightarrow E + T \mid E - T \mid T$ ).
- **Comportamiento:** El sistema gestiona una pila de memoria explícita. En cada paso de evaluación, la traza muestra si la máquina de estados decide realizar un "SHIFT" (leer el siguiente símbolo) o un "REDUCE" (agrupar símbolos leídos), evidenciando de forma impecable la construcción desde las hojas hacia la raíz.
- *Nota Técnica:* Aunque en entornos de producción herramientas como Bison generan automáticamente las complejas tablas de estados (ACTION / GOTO) para un analizador LR(1)/SLR(1), para este ejercicio didáctico se diseñó el algoritmo de precedencia a mano, una técnica clásica para comprender la lógica ascendente.

## Conclusión de la Prueba

Al someter a ambos analizadores a las mismas pruebas de estrés matemáticas, como las entradas  $3 + 4 * 2 - 1$  y  $(1 + 2) * (3 - 4) / 5$ , **ambos produjeron resultados numéricos absolutamente idénticos** (10.0 y -0.6, respectivamente).

Esto demuestra de manera fehaciente que, a pesar de partir de gramáticas estructuradas de forma distinta y emplear motores de lectura completamente opuestos (de arriba a abajo vs. de abajo a arriba), los modelos **LL y LR son dos caminos metodológicos perfectamente válidos para llegar al mismo Árbol de Sintaxis Abstracta (AST)** y, por ende, al mismo significado semántico del código.

## Generadores de analizadores sintácticos y manejo de errores

El proceso de conversión de código fuente a un Árbol de Sintaxis Abstracta (AST) suele apoyarse en herramientas generadoras de parsers. Estas herramientas procesan gramáticas formales y automatizan la creación del analizador, incluyendo mecanismos para gestionar los errores sintácticos que comete el usuario al programar.

## Análisis de Herramientas Destacadas

- **Flex (Fast Lexical Analyzer):** Estrictamente hablando, no es un analizador sintáctico (parser), sino un **analizador léxico** basado en expresiones regulares y autómatas finitos. Su función es identificar los tokens iniciales. Maneja los errores reportando caracteres inválidos mediante reglas por defecto o explícitas en C, delegando la recuperación de errores sintácticos al parser posterior.
- **Bison / Yacc:** Son generadores de parsers ascendentes de tipo **LALR(1)**. Reciben gramáticas libres de contexto (GLC) escritas en notación BNF con acciones integradas en C. Para la gestión de fallos, emplean el token especial error, descartando símbolos hasta sincronizarse con una regla válida (recuperación en modo pánico) y notificando el fallo mediante la función `yyerror()`.
- **ANTLR (v4):** Generador de parsers descendentes que utiliza un algoritmo **LL(\*) adaptativo** (ALL(\*)). Permite definir la gramática completa (léxico + sintaxis) en archivos `.g4` usando notación EBNF. Su manejo de errores se apoya en interfaces como `ErrorListener` y `ErrorStrategy`, capaces de realizar recuperación en modo pánico mediante conjuntos FOLLOW o incluso mediante la inserción/eliminación automática de tokens para continuar el análisis.
- **PLY (Python Lex-Yacc):** Implementación en Python de la arquitectura tradicional Lex/Yacc, construyendo analizadores ascendentes **LALR(1)**. Las reglas de la gramática se especifican directamente en las cadenas de documentación (*docstrings*) de funciones Python en formato BNF. Utiliza la función personalizada `p_error(p)` para capturar fallos y el método `parser.errok()` para recuperarse de ellos.
- **JavaCC (Java Compiler Compiler):** Creador de parsers descendentes **LL(k)** nativo para el entorno Java. Recibe gramáticas EBNF enriquecidas con producciones en código Java. Cuando detecta un error de sintaxis, lanza excepciones de tipo `ParseException` o `TokenMgrError`, indicando la ubicación exacta (línea y columna) y la lista de tokens que se esperaban.

| Generador / Herramienta | Tipo de parser                                            | Gramatica de entrada                       | Mecanismo de manejo y recuperación de Errores                                           |
|-------------------------|-----------------------------------------------------------|--------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Flex</b>             | Generador Lexico (Automatas Finitos / RegEx)              | Reglas RegEx<br>Acciones en C              | Reglas explícitas/por defecto para caracteres no válidos; delega la sintaxis al parser. |
| <b>Bison / Yacc</b>     | LALR(1)<br>(Ascendente)                                   | GLC en notación BNF con acciones en C      | Uso del token error para sincronización en modo pánico y función yyerror().             |
| <b>ANTLR (v4)</b>       | <b>LL(*) Adaptativo</b><br>( <i>ALL()</i> , Descendente)* | Gramática EBNF combinada (.g4)             | ErrorListener / ErrorStrategy con sincronización FOLLOW e inserción/borrado de tokens.  |
| <b>PLY (Python)</b>     | <b>LALR(1)</b><br>( <i>Ascendente tipo Yacc</i> )         | Funciones Python con <i>docstrings</i> BNF | Función p_error(p) personalizada y llamada a parser.errok() para recuperación.          |
| <b>JavaCC</b>           | <b>LL(k)</b> ( <i>Recursivo Descendente</i> )             | Gramática EBNF con producciones Java       | Lanza ParseException / TokenMgrError detallando fila, columna y tokens esperados.       |